

2024年度コンパイラ講義資料

結縁祥治

注意

本資料の使用は，名古屋大学情報学部コンピュータ科学科 3 年生対象講義「コンパイラ」に限る．

謝辞

本資料は以下をもとにして書かれている．

大山口通夫・三橋一郎
コンパイラの理論と作成技法
ISBN978-4-7819-1251-6
サイエンス社

2024 年 10 月現在，当該書籍が絶版となり重版の見込みがたたないことから，本講義の実施にあたり適宜内容の引用・参照をご了承いただいた上記書籍の著者 (大山口先生，三橋先生) に感謝いたします．

第1章 序論

1.1 プログラミング言語と言語処理系

計算機はあらかじめ決まった手順で計算を実行して答えを得る。手順のことを**プログラム** (program) とよび、プログラムを作成することをプログラミングという。計算機のプログラムは決まった書き方で記述する必要があるため、**プログラミング言語** (programming language) によって定める。プログラムは明確に計算機の手順を指定する必要があるため、解釈に曖昧さのないことが望ましい。このため、プログラミング言語は、曖昧性があつたり文脈に依存したりする解釈が難しい人間が使う言語（自然言語）でなく、より機械的に解釈ができる**形式言語**として定められる。

形式言語であっても自然言語にうまく対応付けることで、人間が手順をそのまま指示することができる。このような人間がそのまま読んで解釈が可能なプログラミング言語は一般に**高級言語**と呼ばれる。高級言語としては BASIC, C, Java, Python などがよく用いられる。計算の手順を適切に示すためには、対象とする計算の目的や計算機の構成に沿って記述することが自然であるため、計算のいろいろな側面に注目した高級言語が提案されている。例えば、C 言語はオペレーティングシステムを記述するために開発され、Java はオブジェクト指向という計算の捉え方に沿ってプログラムを記述できる。

これに対して、計算機のハードウェアを直接操作する手順を記述するプログラミング言語は、**低級言語**と呼ばれ、ハードウェアごとに定める必要がある。低級言語の中で直接、ハードウェアを操作する言語（命令列）のことを**機械語** (machine language) という。機械語は 0,1 の並びであるため、普通は人間が理解することは難しい。機械語の命令にわかりやすい名前をつけたり、メモリ領域に対して、ラベルをつけたりする機能をもたせて多少機械語のプログラムに意味を記述できるようにした**アセンブリ言語** (assembly language) がある。アセンブリ言語はハードウェア毎に存在する。アセンブリ言語のプログラムを機械語に変換するコンパイラは**アセンブラ** (assembler) と呼ばれる。

求める計算を計算機を効率的に実行させるためには、計算を実現するプログラムをプログラミング言語で記述する必要がある。人間が手順を指定するためには高級言語で記述するのが都合よい。これに対して、計算機が手順に従って計算を行うためには低級言語で操作を指定する必要がある。高級言語をなんらかの形で計算機で実行するプログラムのことを**言語処理系** (language processor) という。

計算の意味を保存したまま、高級言語から低級言語に変換すれば、効率的にプログラムを作成して実行できる。この変換を行うことを**コンパイル** (compile) といい、変換を自動的に行う計算機プログラムを**コンパイラ** (compiler) という。変換される前プログラムを**原始プログラム** (source program)、変換の結果得られるプログラムを**目的プログラム** (object program あるいは target program) という。人間 (プログラマー) は自然言語（主に英語）に近い形式で書かれた解釈可能な

原始プログラムで手順を記述し、コンパイラで計算機ハードウェアが直接実行可能な低級言語のプログラムに変換することで、目的となるプログラムを実行することができる。コンパイラは原始プログラムを全部読み込み、目的プログラムに変換するのが普通である。

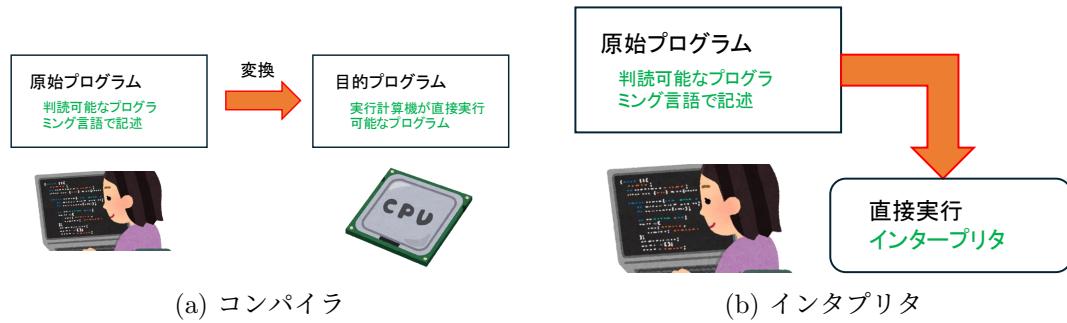


図 1.1: プログラミング言語処理系

これに対して、原始プログラムの手順としての解釈を逐次的に直接的に計算機で実行することによって原始プログラムを計算機で実行することもできる。このような実行プログラムを**インタプリタ** (interpreter) という。インタプリタは、原始プログラムの意味を直接的に実行する。インタプリタは原始プログラムを低級言語に変換することなく、直接的に解釈実行する言語処理系である。

コンパイラとインタプリタもいずれも原始プログラムを実行するための言語処理系である。コンパイラによる実行は原始プログラムを目的プログラムに変換して目的プログラムを実行する。これに対してインタプリタは原始プログラムを直接実行するようなプログラムである。インタプリタは原始プログラムを逐次的に実行するため、一般に実行は低速であるが、原始プログラムを変更すればすぐに変更が実行に反映される。これに対して、コンパイラによって変換された目的プログラムは高速に実行される。コンパイラによる実行は原始プログラムを変更した場合には、再度原始プログラム全体を目的プログラムに変換してから実行し直す必要がある。目的プログラムへ変換するとき、原始プログラムの計算をより効率的に実行するための**最適化** (optimization) が行われる。プログラムに不具合がある場合、インタプリタでは不具合は直接的に実行に反映されるが、コンパイラでは最適化の影響で不具合を特定することが難しい場合もある。プログラムの規模が大きい場合、インタプリタによる逐次的な解釈実行は現実的ではないが、小さなプログラムであれば柔軟に変更できる。Javaなどの言語処理系では、**JIT (Just-in-Time) コンパイラ**が導入され、インタプリタに近い柔軟な実行と目的プログラムによる効率的な実行を組み合わせる言語処理系も存在する。

コンパイラによる変換においては、計算の意味を保存する必要がある。つまり、原始プログラムの計算と目的プログラムの計算は同じでなければならない。しかし、原始プログラムの計算は高級プログラミング言語で定義され、目的プログラムの計算は計算機の機械的な振る舞いで定義される。コンパイラによるプログラミングではこの**計算の意味**を保存することが必須条件である。したがって、コンパイラではプログラムが持つ意味をなんらかの形で正確に表現する必要がある。この意味表現に基づいて意味を保ちながらより効率的な変換である最適化が実現できる。コンパイラを構成するためには、プログラミング言語とプログラムの意味について正確に定式化する必要が生じる。

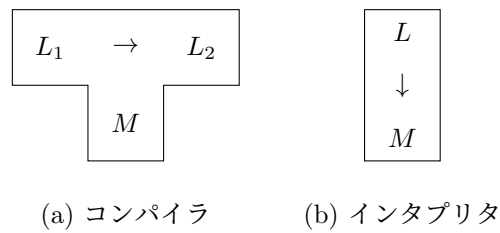


図 1.2: T 図形

言語処理系の構成を示す手法として、**T 図形**が用いられることがある。図 1.2 に T 図式を示す。(a) は言語 L_1 を L_2 に変換するコンパイラが言語 M で記述されていることを示し、(b) は言語 L を M で動作するインタプリタを示す。

例 1 (図 1.3(a)): C 言語のプログラムを Arm 計算機の目的プログラムに変換するコンパイラが Python で記述されているとする。X64 計算機で実行する Python インタプリタがあるとき、C 言語プログラムを X64 計算機で Arm の目的プログラムに変換することができる。

例 2 (図 1.3(b)): C 言語のプログラムを Arm 計算機の目的プログラムに変換するコンパイラが C 言語で記述されているとする。X64 計算機で動作する C 言語を X64 の目的プログラムに変換するコンパイラがあるとき、X64 計算機で C 言語のプログラムを Arm 計算機の目的プログラムに変換するコンパイラを構成することができる。さらにこのコンパイラを最初の T 図式に適用すると、Arm 計算機上で動作するコンパイラを構成することができる。

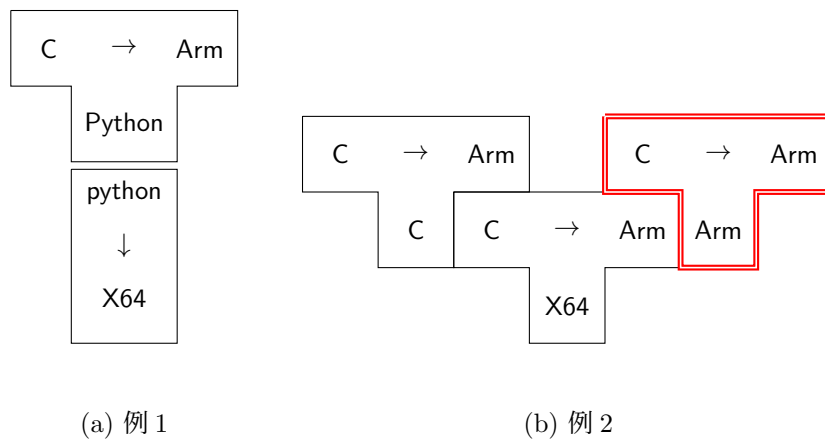


図 1.3: 変換過程

1.2 プログラミング言語の種類

プログラミング言語は計算の手順をどういった側面に注目して記述するかによっていくつかの種類がある。

手続き型言語: 計算の手順を順番に沿って命令の列として記述するプログラミング言語として最も標準的な記述を与える。条件分岐、ループといった制御構造の他に、再帰呼び出し、データ局所化、データ型定義などプログラムの対象に応じた命令、データ型を持つ。

関数型言語: ML(meta language)をはじめとする関数定義に沿ってプログラムを記述する言語である。関数は、入出力の対応を直接的に表す数学のモデルであり、プログラムの最も基本的な意味を表現する。コンパイラは関数の定義に従った計算方法を構築して目的コードに変換する。関数型言語では変換する際に、関数の型の無矛盾性について厳密にチェックすることが多い。厳密な型チェックによって矛盾した定義による不具合の発生を抑えることができる。関数の合成に基づいて、小規模な関数を合成して全体の関数を構成するという考え方に基づく。

論理型言語: 論理における証明が計算に対応するというアイデアに基づいて、ホーン節の反駁によってプログラムを計算する。Prolog が代表的な論理型言語である。証明という概念に基づいていることから、不具合が発生すること＝証明ができず正しくない、という考え方で設計される。

オブジェクト指向言語: 計算の実行主体をオブジェクトとし、オブジェクトにメッセージを送ることで計算が進行するというモデルに基づく。Java がこのアプローチをとっている。オブジェクトという概念は他の概念とは直行したモデルであるので、他の種類の言語でもオブジェクトに基づいた記述を導入している。(Python や caml など) オブジェクト指向モデルによって、データのフローを局所化でき、継承に基づいた合成的な記述が可能となる。smalltalk が本格的なオブジェクト指向言語として提案された。

プログラミング言語は、構文 (syntax) の定義として文脈自由文法を用いることが一般的である。文脈自由文法の記法として、BNF(Backus Naur Form) が用いられる。BNF は、文脈自由文法の生成規則を記述する。生成規則の $\rightarrow$ を $::=$ と記載すること、同じ左辺を持つ生成規則は $|$ によって書くことなどが記法が異なる。

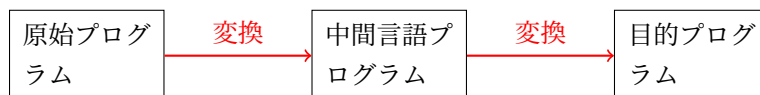
BNF では右辺に $*$ を含めた正規表現を許す。 $*$ が用いられた場合、拡張バックスナーフ形式 (e-BNF) と呼ばれることが多い。文脈自由文法においてはメンバーシップ問題 $w \in L(G)$ は決定可能であることから、文脈自由文法で定義することにより、構文エラーは多項式時間 $(O)(n^3)$ でチェック可能である。

1.3 中間言語

コンパイラがプログラムを変換する際に一段階で変換する場合もあるが、段階を分けて変換していく場合が多い。中間的に変換される対象となる言語のことを**中間言語** (intermediate language) という。

中間言語としては、バイトコードと呼ばれるスタックマシンコードや多くのアーキテクチャに共通な特徴を持つ3番地コードなどが用いられる。中間言語はアーキテクチャに依存しない場合が多く、データフローなどプログラムそのものの性質を解析し、このレベルで最適化を行ったのち、アセンブリ言語に変換することによって、効率的に最適化を行うことができる。Java 言語処理系で

は、まず、バイトコードを目的プログラムとして変換し、バイトコードの実行状況に応じて、バイトコードをさらにコンパイルして、より高速に実行することが可能になる。



3 番地コードは、命令コードと 3 つのメモリ番地を基本命令とする中間言語である。命令コードは直接機械語を対応しており、3 つメモリ番地のうち 1 つをデータの格納先、残り 2 つを計算のデータが収容されている番地を表す。3 番地コードは、ハードウェアに依存しないプログラムの持つ制御フローとデータフローを表現することができる。この特徴から、計算機のアーキテクチャに依存しない基本的な最適化を適用することができる。

具体的な 3 番地コードについては、Java バイトコードや LLVM-IR などがある。

1.4 コンパイラ構成に必要な諸定義

1.4.1 語と集合演算

記号の集合をアルファベットと呼び、 Σ や Γ で表す。 Σ の要素を 2 個並べることによって得られる集合を $\Sigma \cdot \Sigma$ と書く。例えば、 $\Sigma = \{a, b\}$ のとき $\Sigma \cdot \Sigma = \{aa, ab, ba, bb\}$ である。

一般に記号の集合 Σ, Γ に対して、積（あるいは接続）を $\Sigma \cdot \Gamma = \{xy \mid x \in \Sigma, y \in \Gamma\}$ と定義する。ここで、 xy を x と y の接続 (concatenation) という。記号の接続を語 (word) と呼ぶ。 n 個の記号の接続 w の長さを $|w| = n$ と書く。長さ 0 の語を ε と書く。 ε に対する語 w の接続は w そのものである。すなわち、 $w \cdot \varepsilon = \varepsilon \cdot w = w$ である。

集合 Σ の要素を i 個連続して接続するところによって得られる集合を Σ^i と書き、以下のように定められる。

$$\Sigma^{i+1} = \begin{cases} \{\varepsilon\} & i = 0 \text{ のとき} \\ \Sigma \cdot \Sigma^i & i \geq 1 \text{ のとき} \end{cases}$$

$\cap, \cup, \setminus$ は、2 つの集合 Σ, Γ に対して以下のようにそれぞれ積集合、和集合、差集合を構成する。

$$\begin{aligned} \Sigma \cap \Gamma &= \{x \mid x \in \Sigma \text{ かつ } x \in \Gamma\} & \Sigma \cup \Gamma &= \{x \mid x \in \Sigma \text{ または } x \in \Gamma\} \\ \Sigma \setminus \Gamma &= \{x \mid x \in \Sigma \text{ かつ } x \notin \Gamma\} \end{aligned}$$

Σ の 1 個以上の接続のすべての集合を Σ^+ と書く。

$$\Sigma^+ = \bigcup_{i=1}^{\infty} \Sigma^i = \Sigma \cup \Sigma^2 \cup \dots \cup \Sigma^n \cup \dots$$

$\{\varepsilon\} \cup \Sigma^+$ を Σ の閉包 (closure) あるいはクリーネ閉包 (Kleene closure) と呼び、 Σ^* と書く。

$u, v \in \Sigma^+$ に対して、 $uw = v$ を満たす $w \in \Sigma^*$ が存在するならば、 u は v の接頭語 (prefix), $wu = v$ を満たす $w \in \Sigma^*$ が存在するならば u は v の設備語 (postfix) であるという。ここで、 $u \cdot \varepsilon = u$ および $\varepsilon \cdot u = u$ であることから、語 u は自身の接頭語でもあり、設備語である。

空集合は $\emptyset$ と記述する. $\emptyset \cup \Sigma = \Sigma \cup \emptyset = \Sigma$, $\emptyset \cap \Sigma = \Sigma \cap \emptyset = \emptyset$ である. 連接の定義から, 任意の $L \subseteq \Sigma^*$ に対して, $\emptyset \cdot L = L \cdot \emptyset = \emptyset$, $\emptyset^* = \{\varepsilon\}$ である.

1.4.2 グラフと木

グラフ (graph) は**頂点** (vertex) の集合 V と**辺** (edge) の集合 $E \subseteq V^2$ の対 (V, E) である. 辺 $e = (v_1, v_2)$ は, 頂点 $v_1 \in V$ から $v_2 \in V$ への矢印 $v_1 \rightarrow v_2$ で表す. v_1 は e の始点, v_2 は e の終点と呼ばれる. 辺の始点と終点を区別するとき, **有向グラフ** (directed graph) という. 辺 (v_1, v_2) と (v_2, v_1) を同じ辺であるとするとき, **無向グラフ** (undirected graph) という. 頂点 u から出る辺の本数を u の**次数** (degree) という. $I \subseteq V$ に対して, $I^2 \cap E = \emptyset$ であるとき, すなわち I に含まれる任意の頂点の間に辺が存在しないとき, I は**独立集合** (independent set) という. 有向グラフにおいて, 辺 $(u, v) \in E$ に対して, u は v の**先行頂点** (predecessor), v は u の**後続頂点** (successor) という. 頂点の系列 $v_1, \dots, v_k$ に対して, $(v_i, v_{i+1}) \in E$ が $1 \leq i \leq k-1$ に対して成り立つとき, グラフの**路** (path) であるという.

空でない頂点集合を持つ有向グラフ $G = (V, E)$ において, $v \in V$ で先行頂点を持たない頂点がただ一つ存在し, それ以外の頂点 $u \in V \setminus \{v\}$ については, ただ1つの先行頂点を持つとき G は**木** (tree) であるという. 先行頂点を持たない頂点のことを**根** (root) という. 木の条件を満たすグラフ T の根を $\text{root}(T)$ で表す. T の頂点のうち, 後続頂点を持たない頂点を**葉** (leaf) という. 木の条件を満たす有向グラフの頂点に対して, 先行頂点を**親** (parent), 後続頂点を**子** (child) という. 根は親頂点を持たず, 葉は子頂点を持たない.

第2章 コンパイラの概要

2.1 コンパイラの構造

コンパイラはプログラマーが記述する原始プログラムを**フェーズ** (phase) と呼ばれるいくつかの段階を経て計算機が直接実行可能な 0,1 の系列である機械語で記述された目的プログラムへ変換する。目的プログラムは**バイナリコード**とも呼ばれる。

コンパイラの標準的な構造を図 2.1 に示す。

字句解析 (lexical analysis) 字句解析は、アルファベットの系列としての原始プログラムを入力として、プログラミング言語で定められる単語に切り分ける。プログラミング言語の多くは基本的な記述として、1 バイトの文字コードを用いる。一般的には ASCII コード (ANSI X3.4-1986) を用いる。プログラミング言語ごとに定められた予約語 (keyword)、識別子 (identifier)、定数 (constant)、文字列リテラル (string literal)、区切り記号 (delimiter) などを**トークン**として認識し、原始プログラムをトークン列に変換する。トークンは有限オートマトンによって定義される。

構文解析 (parsing) 構文解析は、字句解析の結果出力をプログラミング言語を定義する文脈自由に従ってプログラムの基本的意味を表現する**解析木** (parse tree) を構成する。解析木は構文木 (syntax tree) ともいう。解析木は、プログラムが構文的に持つ情報を表現する。

中間コード生成 (intermediate code generation) 構文木はプログラミング言語が記述するプログラムの意味を表し、構文木をトラバースすることによってプログラムが記述する操作を抽出することができる。計算機ハードウェアに依存しない制御構造とデータフローを中間コード言語のプログラムで表す。中間コードとして、スタックマシンの命令であるバイトコードや3番地コードが用いられる。中間コードは機械語の目的コードを生成するための前段階であり、**フロントエンド** (front end) と呼ばれる。

コード最適化 (code optimization) 解析木から生成される中間コードはプログラムの持つ意味を直接的に表す。しかし、重複した操作が繰り返し実行されるなどの冗長な操作を含む。計算の意味を保存しながらより効率的なコードを生成することを**コード最適化** (code optimization) という。コード最適化は、計算の意味を保存することが要求される。コード最適化は計算の意味を保たれる範囲でより効率的なコードを発見的に生成する。このため、最適化されたコードが必ずしも最も効率的であることは保証されない。最も効率的なコードを得ることは計算理論の結果から不可能であることが示される。

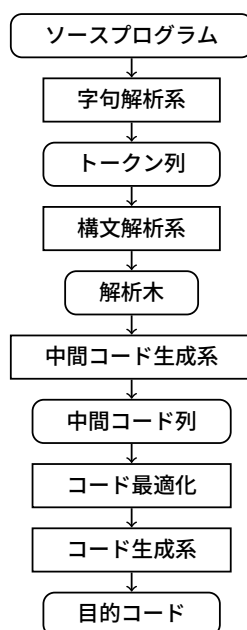


図 2.1: コンパイラの構造

目的コード生成 (object code generation) 目的コード生成では、最適化された中間コード言語のプログラムを機械語のプログラムに変換する。このフェーズでは、中間コードが表す制御フローとデータフローに従って目的コードを実行する計算機の資源を効率的に利用する機械語コードを生成する。計算機ハードウェアでは、演算を実施するためのデータの主記憶からの転送をできるだけ少なくする必要がある。演算のためのレジスタの数はプログラムで利用できるメモリの数は非常に少ないため、プログラムを解析することで効率よくレジスタを利用する機械語プログラムを生成する必要がある。プログラムの操作に対してハードウェアで用意されている効率的な命令を選択する必要がある。

2.1.1 コンパイラにおけるパス

原始プログラムに含まれる表現を一度だけ読んで処理を行う部分を **1パス** (pass) という。コンパイラにおけるいくつかのフェーズは一つのパスにまとめることができ、例えば、字句解析と構文解析は一つのフェーズにまとめることができる。目的コードを原始プログラムを一度だけ読んで生成するコンパイラのことを **1パスコンパイラ** (one-pass compiler) という。実際的なコンパイラは多パスであるものがほとんどである。

2.2 字句解析の概要

字句解析は原始プログラムの文字列をプログラムとして意味のある単語にまとめてトークンの系列として出力する。プログラミング言語で用いられるトークンの種類は以下のようなものがある。

- **予約語 (keyword):** プログラミング言語によって特別な意味を持つ単語。条件分岐を構成する, `if`, `then`, `else` や, 繰返し構造を構成する `while`, `do` などのほかに, 変数に対する組み込みのデータ型を表す `int`, `char` がある。これらの語はプログラミング言語が決まった意味で用いるために特に区別して認識する。
- **識別子 (identifier):** プログラムで用いる変数名, 関数名などに用いる文字系列である。通常, 英数字と一部の記号の系列で構成される。
- **定数 (constant):** 先頭の符号と数字および小数点で構成される数 (例: `-123.4`) や浮動小数点表現 (例: `3.14E1`)。一つの文字 'A' も定数として認識する。
- **文字列リテラル (string literal):** ダブル引用"で囲った文字の並び。"Hello World"のように空白も含めることも可能。
- **区切り子 (delimiter):** プログラムの構文要素を区切る記号。セミコロン `;` は逐次に行う命令の区切りを表す。このほか, 並びの区切りを表すカンマ `,` や括弧 `()`, `{}`, `[]` がある。数式, 論理式において数字や記号を区切る記号 `+`, `-`, `&&`, `=`, `>=` がある。

トークンは, 正規表現を用いて定義することができる。このため, 字句解析を行うためには, トークンを受理する有限オートマトンを構成して, 有限オートマトンの受理系列をトークンとして出力する。

2.3 構文解析の概要

一般にプログラミング言語は文脈自由言語として定義される。このため, プログラミング言語を与える文脈自由文法 G が存在し, 原始プログラムをトークン列に変換した語 w が G の生成する言語に属するかどうか $w \in L(G)$ を判定する。

$w \in L(G)$ のとき, 原始プログラムはプログラミング言語としては誤りがない。このとき, G の開始記号 S から w に至る導出が存在し, 文脈自由言語の特徴から導出の過程は木構造で表すことができる。この木構造は, プログラムの基本的な意味を表現する。例えば, 以下のように文脈自由文法 G を与える。

$$G = \langle \{+, n\}, \{S, E\}, \left\{ \begin{array}{l} S \rightarrow E \\ E \rightarrow E+E \\ E \rightarrow n \end{array} \right\}, S \rangle$$

このとき, $n+n+n \in L(G)$ である。これは, $S \Rightarrow E+E \Rightarrow n+E \Rightarrow n+E+E \Rightarrow n+n+E \Rightarrow n+n+n$ という導出系列が存在するためである。ここで n は自然数に対するトークンであり, 実際には `1, 12, 30` などの数字を字句解析した結果である。

この導出に対する解析木は図 2.2(a) のようになる。

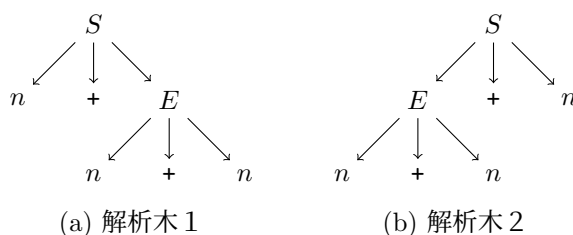


図 2.2: 導出に対する解析木

$n+n+n$ には、別の導出 $S \Rightarrow E + E \Rightarrow E+n \Rightarrow E+E+n \Rightarrow E+n+n \Rightarrow n+n+n$ も存在する。この導出に対応する解析木は (b) のようになる。この導出は $E+E$ において、右の E を n に置き換えてから、左の E を $E+E$ に置き換えている。(a) では $E+E$ の左の E を n に置き換えてから、右の E を $E+E$ に置き換えている。(a) の解釈では、 $+$ を加算とすると、右の 2 つの n を足してから左の n を加算する。(b) の解釈では、左の 2 つの n を足してから右の n を加算する。

$w \notin L(G)$ であれば、 w は文法エラーを含む。通常の言語処理系の構文解析では、単にエラーであることを通知するだけでなく、部分的な構文木を作成して、エラーを起こしている理由をプログラムに提示する。例えば、 $n++n \notin L(G)$ である。最初の $+$ に対して木を構成しようとする $E+E$ の右側の E を $+$ に置き換えることができないため、エラーが生じ、 $+$ の右辺が間違っているといったエラーメッセージが出力される。

コンパイラ構成の観点からプログラミング言語の構文解析に対してはより実用的な要求がある。解析木はプログラムの意味に対応するため、曖昧でない文法が必要になる。曖昧な文法で定義する場合は、どの解析木を導出の結果とするかについてあらかじめ決めておく必要がある。上の文法 G はあいまいで $n+n+n$ に対して図 2.2 の 2 つの解析木が存在する。加算演算は結合的であるので、どちらの木であっても答えは変わらない。しかし、 $+$ を $-$ に置き換えて減算とすると、減算は結合的でないので答えが定まらない。このため、通常は解析木 (b) のように解釈することが決められている。3-2-1 は通常 0 と評価される。解析木 (a) のように解釈すると 2 と評価される。

文脈自由文法 G に対して、 $w \in G$ は決定可能な問題であり、入力長さに対して $O(n^3)$ で決定することができるアルゴリズムが知られている。コンパイラの構成上、より早い構文解析が必要であり、より高速な構文解析アルゴリズムに合わせてプログラミング言語を定義する文脈自由文法を決定する。決定的に構文解析するための文法のクラスが定義される。

2.4 中間コード生成の概要

中間コードは原始プログラムが持つ基本的な振舞いを抽象的に表現する。中間コードは原始プログラムの手順を表す実行制御フローとデータに対する演算とメモリの更新を表すデータフローを表す。中間言語として、**逆ポーランド記法** (postfix Polish notation あるいは reverse Polish notation)、**抽象構文木** (abstract syntax tree)、**3 番地コード** (three-address code)、バイトコードなどが用いられる。

中間言語が表す振舞いは、プログラムが表す計算手順を標準的な命令単位で並べたものであり、計算機ハードウェアから独立している。中間言語に一旦変換することで、目的プログラムが持つ本

質的な動作意味を単純に表現することができる。このことから異なるアーキテクチャの目的コードは中間コードを変換して生成することができ、移植性が高くなる。

中間言語プログラムでは、変数は**記号表**を作成して実際に割り当てるメモリ番地に変換する。中間言語プログラムの実行環境では、**駆動レコード**とよばれるスタックによって、変数の値を格納する番地を決定する。逆ポーランド記法やバイトコードではデータスタックによってメモリの格納番地を決定する。

このフェーズでは変数の格納番地の決定を行うとともに、データ型など解析木に対してより詳細な制約をチェックする**意味解析**を行う。例えば、 $x+y$ という式に対して、 x と y が同じ型であるかどうか、違う型であっても加算演算が可能かどうかといった**型チェック**を行う。

2.5 コード最適化の概要

コード最適化 (code optimization) は計算の意味を変化させない範囲でより短くて高速な命令系列を導くことである。コード最適化は中間言語プログラムに対して行われるものと目的プログラムに対して行われるものがある。コード最適化は命令を制御が分割したり合流したりしない**基本ブロック**に対して行う**局所最適化** (local optimization) (あるいは**覗き穴最適化** (peep hole optimization) とも呼ばれる)、基本ブロックより広い範囲で行う**大域最適化**がある。その他、数値計算などにおいて最も効果が高い繰り返しループに着目して行う**ループ最適化**がある。一つの手続き範囲 (intra-procedure) で行う最適化、さらに、手続きをまたいで行う (inter-procedure) 最適化がある。最適化は範囲が広がるほど、意味を変えずに効率化することが難しくなる。最適化によって、原始プログラムと目的プログラムの対応をとることが難しくなるので、原始プログラムをデバッグする場合は、最適化を行わない場合が多い。

例えば、図 2.3(a) の中間言語プログラムの断片に対して、同じ計算が繰り返されているため、(b) のように最適化することができる。(a)、(b) は変数に同じ値を与えると各変数に同じ値を返すので、計算の意味を保存する。

$x=1$	$x=1$
$z=x+y$	$z=1+y$
$w=x+y$	$w=z$
$u=z*4$	$u=z*4$
$v=w*4$	$v=u$
(a) 最適化前	(b) 最適化後

図 2.3: 簡単な最適化の例

大域的な最適化においては**データフロー解析**あるいは**静的単一代入形式**などの方法が用いられる。

2.6 目的コード生成の概要

中間言語コードから実際に実行対象となる計算機ハードウェアが直接実行できる機械語を生成する。目的コード生成では中間言語プログラムの操作をハードウェアの適切な命令に置き換えることと中間コードのメモリをデータ演算を行う際に使用するレジスタに対応させる。ここでは、レジスタの値をできるだけ変更せず、メモリとレジスタの転送を少なくする**レジスタ割り当て**を行う必要がある。

3 番地コードの並び

```
t1=i2-i3
t2=i1+t1
```

を機械語に対応するアセンブリコードにそのまま変換すると以下のようになる。

```
Load   i2
Sub     i3    } t1=i2-i3
Store  t1
Load   i1
Add     t1    } t2=i2+t1
Store  t2
```

ここで、2つめの3番地コードを $t2=t1+i1$ とおきかえてもよい。すると、最初の命令で計算した $t1$ をすぐにメモリに格納する必要はなく、以下のように命令を一つ省略できる。

```
Load   i2
Sub     i3
Store  t1
Add     i1
Store  t2
```

さらに $t1$ がこのあと参照されないことがわかればさらに命令を省略できる。

```
Load   i2
Sub     i3
Add     i1
Store  t2
```

変数の参照と更新に着目すれば、ハードウェアで使用するレジスタの値とメモリとの転送命令 `Load`, `Store` を減らして高速化することが可能である。与えられたプログラムに対してレジスタ割り当てを最適にする問題は、有限の組み合わせに対してプログラムを解析することで可解である。しかし、そのアルゴリズムの時間計算量は NP 完全であることが知られているため、レジスタ割り当てによって実行時間を短縮できる効果よりも割当の組み合わせを計算する時間の方が長くなってしまうことが考えられる。このため、実際のコンパイラではより計算量の小さな近似アルゴリズムが用いられる。

第3章 字句解析

3.1 正規表現

正規表現は指定された語の集合（言語）を指定する．コンパイラにおいては，トークンの種類を正確に指定するために用いる．予約語，識別子，定数，文字列リテラル，区切り子を指定する正規表現を決める．

識別子 (identifier) として認められる文字系列は，「非数字または被数字の後に非数字か数字を複数個連接した系列」として定義される．数字文字 Dec, 非数字文字 NDec, 識別子は ID は次の正規表現で定義することができる．

$$\begin{aligned}\text{Dec} &::= 0|1\cdots|9 \\ \text{NDec} &::= _|\text{a}|\cdots|\text{z}|\text{A}|\cdots|\text{Z} \\ \text{ID} &::= \text{NDec}(\text{NDec}|\text{Dec})^*\end{aligned}$$

10 進数定数 Decimal は以下のように定義できる．

$$\begin{aligned}\text{NZDec} &::= 1|\cdots|9 \\ \text{Decimal} &::= \text{NZDec}\text{Dec}^*\end{aligned}$$

正規表現は次のように定義される．

定義 3.1. 集合 Σ 上の**正規表現**は以下の再帰的定義によって与えられる表現である．

- (1) $\varepsilon, \emptyset$ は正規表現である．
- (2) Σ の要素 $a \in \Sigma$ は正規表現である．
- (3) R_1, R_2 が正規表現であるとき， $R_1|R_2, R_1R_2$ は正規表現である．
- (4) R が正規表現であるとき， R^* は正規表現である．

「再帰的定義」は，自らの定義の中に自身が含まれている定義である．上の定義では，(3),(4) において，正規表現の定義に正規表現自体が使われている．ここでは，定義しようとしている表現よりも短い表現が定義の中で使われているため，うまく定義されていることに注意しよう．例えば， $\Sigma = \{a, b\}$ に対して $(ab)|(b^*)$ が Σ 上の正規表現であるためには，(3) から ab と b^* が正規表現でなければならない． ab が正規表現であるためには，(3) から a, b が正規表現でなければならない． $a, b \in \Sigma$ なので， a, b は (2) により正規表現である． b^* が正規表現であるためには b が正規表現でなければならないが， $b \in \Sigma$ なので正規表現である．したがって， $a|(b^*)$ は正規表現である．括弧はまとまりを表すために適宜用いる．演算の結合の順番は強い順に $*$, 接続, $|$ とする． $ab|b^*$ は $(ab)|(b^*)$ を表す．

定義 3.2. Σ 上の正規表現 R に対して, R が表す系列の集合 $\llbracket R \rrbracket \subseteq \Sigma^*$ を以下のように再帰的に定める.

- (1) $\llbracket \varepsilon \rrbracket = \{\varepsilon\}, \llbracket \emptyset \rrbracket = \emptyset$
- (2) $a \in \Sigma$ に対して $\llbracket a \rrbracket = \{a\}$
- (3) $\llbracket R_1 R_2 \rrbracket = \llbracket R_1 \rrbracket \cdot \llbracket R_2 \rrbracket, \llbracket R_1 \mid R_2 \rrbracket = \llbracket R_1 \rrbracket \cup \llbracket R_2 \rrbracket$
- (4) $\llbracket R^* \rrbracket = \llbracket R \rrbracket^*$

ここで右辺に現れる $\cdot, *$ は 1.4 節で定義した集合に対する接続と閉包を表す.

従って, $ab|b^*$ という正規表現が表す Σ の系列の集合は以下ようになる.

$$\begin{aligned}
 \llbracket ab|b^* \rrbracket &= \llbracket ab \rrbracket \cup \llbracket b^* \rrbracket \\
 &= \llbracket a \rrbracket \llbracket b \rrbracket \cup \llbracket b \rrbracket^* \\
 &= (\{a\} \cdot \{b\}) \cup \{b\}^* \\
 &= \{ab\} \cup \{\varepsilon, b, bb, bbb, \dots\}
 \end{aligned}$$

プログラミング言語のトークンは正規表現で定義される.

3.2 有限オートマトン

正規表現によって表現される文字列を認識するメカニズムは有限オートマトンである. 有限オートマトンは, 有限の状態を持つ状態遷移機械であり, ある状態 q_1 から次の状態 q_2 へ遷移するとき Σ の要素 a を読む. この遷移を $q_1 \xrightarrow{a} q_2$ と書く. 初期状態から, 受理状態に至る遷移の系列で受理する文字系列がオートマトンが受理する語である. q_0 が初期状態で, $q_0 \xrightarrow{a_1} q_1, q_1 \xrightarrow{a_2} q_2, \dots, q_{n-1} \xrightarrow{a_n} q_n$ という遷移があり, q_n が受理状態であるとき, 語 $w = a_1 a_2 \dots a_n$ をオートマトンが受理するという.

定義 3.3. 有限オートマトンは以下の5項組で定義される. $\langle Q, \Sigma, \delta, q_0, F \rangle$ ここで, Q は状態の有限集合, δ は遷移関係, $q_0 \in Q$ は初期状態, $F \subseteq Q$ は受理状態集合である.

δ は3つ組 (q, a, q') の集合であり, q から a を読んで q' に遷移することを示す. 遷移関係は $(q, a, q') \in \delta$ を $q \xrightarrow{a} q'$ のように表す. 遷移関係 δ を Σ^* に以下のように拡張した関係 $q \xRightarrow{s} q'$ に拡張する.

$$\begin{aligned}
 q &\xRightarrow{\varepsilon} q \\
 q &\xRightarrow{as} q' \quad \text{ここで, } q \xrightarrow{a} q'', q'' \xRightarrow{s} q'
 \end{aligned}$$

定義 3.4. 有限オートマトン $A = \langle Q, \Sigma, \delta, q_0, F \rangle$ に対して $L(A) = \{w \mid q_0 \xRightarrow{w} q_f, q_f \in F\}$ を A の受理言語という.

有限オートマトンの受理言語と正規表現が表す言語には以下のようなよく知られた関係がある.

定理 3.1. • 正規表現 R に対して $\llbracket R \rrbracket$ を受理する有限オートマトン A_R が存在する.

- 有限オートマトン A に対して $L(A) = \llbracket R \rrbracket$ となる正規表現 R が存在する.

このことから、正規表現と有限オートマトンは表現能力が等しいことがわかる。また、有限オートマトンは、受理言語の積、和、否定に関して閉じていることが知られている。

定理 3.2. • 有限オートマトン A_1, A_2 に対して、 $L(A_3) = L(A_1) \cap L(A_2)$, $L(A_4) = L(A_1) \cup L(A_2)$ を満たす有限オートマトン A_3, A_4 が存在する。

- 有限オートマトン A に対して、 $L(A_c) = \Sigma^* \setminus L(A)$ を満たす有限オートマトン A_c が存在する。

上記定理の証明については、オートマトンの教科書¹を参照してほしい。

3.3 字句解析の仕組み

字句解析では、原始プログラムを Σ の系列として受取り、トークンごとに切り分けて出力する。トークンの種類に応じてオートマトンを構成する。それぞれのオートマトンを A_{kw} , A_{id} , A_{cnst} , A_{str} , A_{del} とする。

字句解析はこれらのオートマトンを図??のように組み合わせて構成される。

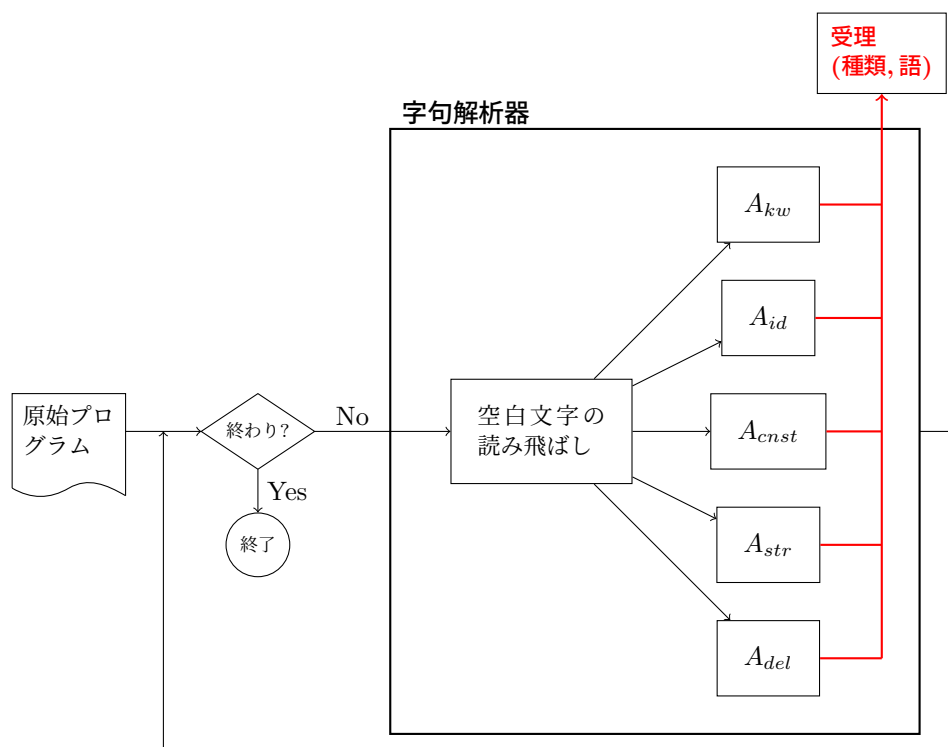


図 3.1: 字句解析器の構成

¹J. ホップクロフト/J. ウルマン:「オートマトン言語理論 計算論 1」訳:野崎昭弘ら 2.5 および 3.2

ステップ1: 空白, タブコード, 改行などの空白文字を読み飛ばす.

ステップ2: 次の文字を A_{kw} , A_{id} , A_{cnst} , A_{str} , A_{del} に入力する.

ステップ3: いずれかのオートマトンが受理状態に達するまで, ステップ2を繰り返す.

ステップ4: いずれかのオートマトンが受理状態に達したらそのオートマトンの種類と受理した系列を出力し, すべてのオートマトンの状態を初期状態に戻す.

ステップ5: 入力 of の最後に達していなければ, ステップ1に戻る. 入力の最後に達したら終了

字句解析の実現においては, トークンの全体の和集合を受理するオートマトンを作成し, 受理状態がどのトークンによるものかを区別しておくことで字句解析が実現ができる. 有限オートマトンが和集合について閉じていることからこのような構成はいつも可能であることが保証されている.

受理状態が2つ以上のオートマトンによる場合は, 受理の優先順位を決めておくことで字句解析を行う. 予約語と識別子は一般に同じ文字列を受理する可能性がある. その場合は, 予約語を優先させる.

字句解析器のプログラミング 字句解析を行うオートマトンを構成するために, 正規表現を用いる. 正規表現 R を $\llbracket R \rrbracket$ を受理する有限オートマトンに変換して, 字句解析器に組み込む. 有限オートマトンからプログラムへの自動変換は比較的容易に実現できる. 字句解析器は, トークンの集合を正規表現で記述するだけで, 自動生成することが可能である.

図3.2に識別子と自然数定数を受理するオートマトンを示す. いずれかを受理するオートマトン $A_{id \cup cnst}$ は(c)のようになる. s_1 が受理状態であった場合は識別子, s_2 が受理状態であった場合は自然数定数である.

3.4 字句解析器の生成

3.4.1 正規表現の記法

オートマトンに対応する動作をさせるために計算機上で正規表現は以下のように記述される.

- ε , $\emptyset$ を直接表現する記法はない.
- $a \in \Sigma$ については, "a" のように引用符で囲む. $s \in \Sigma^+$ についてはまとめて "abc" のように引用符で囲む.
- $R_1 | R_2$ については, R_1 の記法 | R_2 の記法. 連続した文字コードを持つ場合には, $[X-Y]$ のように途中を省略できる. $s = a_1 a_2 \dots a_n \in \Sigma^+$ に対して $[s]$ は $a_1 | a_2 | \dots | a_n$ を表す. 例えば $[0-9]$ は $0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9$ を表す.
- Kleene 閉包は $*$ を後ろにつける. $[ab]^*$ など
- ラベルの後ろに正規表現を書いて名前をつけることができる. NZDEC $[1-9]$ や DEC $[0-9]$ など. ラベルの参照時は $\{\}$ をつける.

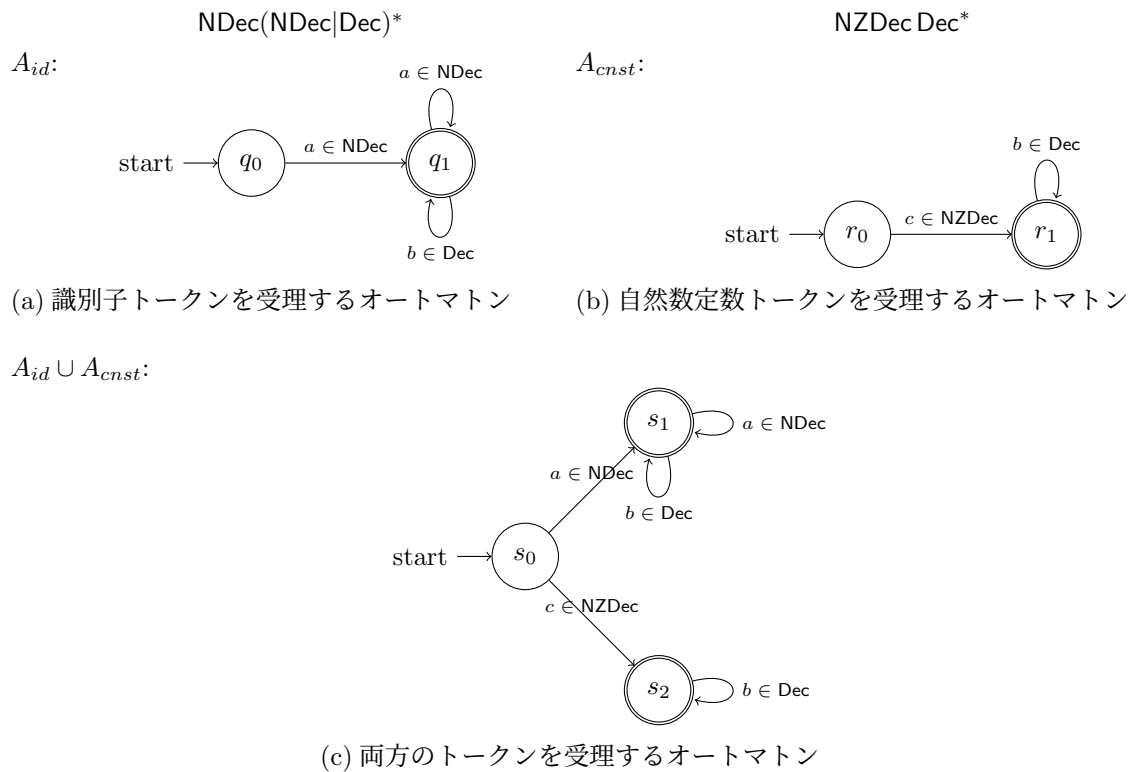


図 3.2: 字句解析の正規表現とオートマトン

ラベルの機能を使うと自然数定数は $\{\text{NZDEC}\}\{\text{DEC}\}^*$ と記述できる。

$\text{NDEC} [_a-zA-Z]$ とすると、識別子名を表す系列は $\{\text{NDEC}\}(\{\text{NDEC}\}|\{\text{DEC}\})^*$ となる。

3.4.2 字句解析器生成ツール

lex

lex では正規表現とアクション (C 言語) の対を並べて記述し、入力文字列から正規表現で構成されるオートマトンが受理する系列を抽出して、対応するアクションを実行するような C 言語プログラムを生成する。受理した入力系列は `yytext` という文字列に格納される。

例えば、以下のような仕様ファイル `lex1.1` を用意したとする。

```
%%
[A-Za-z]+ {printf("String found:%s\n",yytext);}
[0-9]+    {printf("Digits found:%s\n",yytext);}
.         {printf("Other character:%d\n",yytext[0]);}
%%
```

lex では、ピリオド "." は Σ の任意の文字のいずれかであることを示す。受理の優先度は上に書かれるほど高い。入力のある "A" は 1 番めと 3 番めの両方に受理されるが、アクションは 1 番めが実行される。lex は、上のような仕様を受け取って、正規表現にマッチする系列が入力されたとき、アクションを実行する C 言語プログラムを生成する。

```
% lex -o lex1.c lex1.l
% cc -o lex1 lex1.c -ll
```

1 行目で仕様 lex1.l から lex1.c を生成し、これを 2 行目で実行形式に変換している。

より複雑なプログラムを書くために最初の %% の前に C 言語の include 文や define マクロまたはプロトタイプ宣言、最後の %% の後に C 言語の関数を記述することができる。

```
%{
    #include <stdio.h>
    void pp(char*);
}%
%%
[A-Za-z]+ {pp("String");}
[0-9]+    {pp("Digits");}
.         {printf("Other character:%d\n",yytext[0]);}
%%
int main(void)
{
    yylex();
    return 0;
}
void pp(char *s) {
    printf("%s found:%s\n",s,yytext);
}
```